# An Empirical Analysis of JavaScript Framework Overhead for NLP-Based Educational Web Applications Under Simulated Classroom Computing Constraints

Kavitharshan Baskaran
Department of Computer Science
University of Warwick
Coventry, England
u5759839@warwick.ac.uk
Supervisor: Dr. Mike Joy

## I. Introduction

With the age of technology already upon us, advances in web-based technology have brought education to a point where machine learning models can be embedded directly into everyday classroom tools, rather than being limited to research prototypes. This has opened the floodgates to a variety of unique educational applications, from tutoring systems to chatbots that help students learn. However, a common yet under-appreciated design decision when building such web-tools is the choice of the front-end JavaScript framework that turns the application from static to interactive. Whilst framework benchmarkers do exist, currently they typically only measure simple, crud operations such as creating list-items and re-ordering DOM nodes, all on developer-grade hardware. This hardly reflects the harsher reality of classrooms, where there are ageing chromebooks, congested school WIFI, and hardware that is several generations behind what frontend developers are accustomed to, all posing significant constraints. Consequently, there is a clear need for empirical evaluation of front-end framework performance under realistic, resource-constrained classroom conditions.

This project goal is to investigate whether the choice of frontend framework measureably affects the performance of a realistic, NLP-driven educational web application when run under simulated classroom constraints. Within this project the frameworks that will be discussed are React amd Angular, using Vanilla JavaScript as a baseline in order to isolate framework overhead. Rather than constructing a generic benchmark, each application will perform an automatic classification of student writing level based on CEFR (Common European Framework of Reference for Languages) proficiency level, this is done by using a pretrained transformer-based classifier. This is to ensure that the performance evaluation is reflective of a meaningful, educationally relevant task rather than artificial benchmarks. It does so by providing realistic asynchronous workload, text submission, network request, model inference and UI update.

The central research question that I will be answering is, does frontend framework choice produce a measurable and practically significant difference in application responsiveness under classroom-represenative hardware and network constraints. Three functionally identical versions of the same application will be built, each in their respective framework implementations. They also share an identical backend and visual design, so any measured performance difference can only be connected specifically to the fronend framework.

## II. Background and Research

To say the field of computer science has not been short of framework comparisons would be an understatement, with existing approaches usually relying on operation level benchmarks rather than realistic application and hardware level workloads. One piece of literature that has been widely referenced is Krauses' 'js-framework-benchmark [1], this is an open source project that measures the duration of isolated DOM operations, including creating, updating, selecting and removing rows in a large table. Currently, the benchmarker features 186 framework implementations which makes it valuable as a low-level comparator. However, this benchmark deliberately isolates rendering performance from real application concerns, explicitly not including backend response handling and asynchronous network request handling. This reveals a critical gap in this current literature, the js-framework-benchmark provides valuable insight to DOM reconcillation speed, real world applications require a more holistic approach to performance assessment. Real applications require full lifecycle performance. A more recent study by Piastou [2] has improved this approach by evaluating 3 distinct frameworks (React, Angular, and Vue.js) against different load conditions or scalability scenarios, it does this by measuring against performance indicators such as load times, speed of rendering,

memory usage, and CPU usage. The results obtained from this research were very compelling, with the paper coming away with various findings, such as Angulars richer feature set increasing load time and memory usage relative to the other implementations. However, this study, like js-framework-benchmark , was conducted on developer grade hardware, and does not provide an accurate representation of how performance differences may occur when under resource-constrained conditions.

In order to limit test framework implementations even further, many developers have studied web applications under constrained hardware and network conditions, though typically outside the context of framework comparison specifically. Pandeys work [3] on MAML documents that growing website complexity, in particular heavy reliance on JavaScript disproportionatly affects users on low end devices, which leads to longer load times and unresponsiveness. They introduce a solution, MAML, which is an alternative lightweight web language that is designed to reduce JavaScript and CSS payload delivered to constrained devices. While this proposed solution addresses this problem by replacing conventional web technologies altogether, it neither asks nor answers a much narrower question which this dissertation poses to question, given that a development team is already commited to using conventional JavaScript frameworks, does the specific choice of framework meaningfully affect the experience of users on constrained devices, and by how much?. Pandey et al's finding tells us that JavaScript payload size is a primary driver of degraded performance on low-end devices also motivates the use of bundle size as a measured metric.

On the NLP side of this project, an automatic CEFR-level classification is used to provide a realistic and educationally meaningful workload. This task has been proven to be well established, with Lagutina et al [4] conducting a comprehensive evaluation of both traditional machine learning classifiers and BERT-based models for this purpose. Their results demonstrate that BERT-based models achieve strong performance in this domain, reaching an F-score of 69% when classifying texts across CEFR levels. Lagutina et al's work also provided insights such as classification errors being more prevelent between neighbouring classes, which makes sense given inherent subjectivity. More broadly, their findings confirm the effectiveness of automatic CEFR classification and its viability for practical deployment in e-learning systems, reinforcing the suitability of this task as a realistic workload for evaluating frontend framework performance. These findings inform the choice of a fine tuned transformer based classifier (`theluantran/cefr-bert-classifier`) as the NLP component of this project.

The works cited above represent the key references that directly frame the research questions and methodological approaches used in this project. However, it should be acknowledged that the scope of this interim report does not provide a fully comprehensive account of all the literature I have come across that which shaped the background understanding and technical approach of the project. The above reviewed literature reveals a clear research gap which this project aims to position itself to address, which is that framework performance comparisons exist, and low-end-device web performance studies exist, but the intersection (i.e. how the choice of frontend framework specifically affects the performance of an NLP application under simulated classroom-representative hardware and network constraints) has not been thoroughly studied.

## III. Where Are You Now?

### A. Technical Content

So far, the core implementations that are required to begin empirical testing have been completed and version controlled on GitHub[1]. These implementations are consistent across framework applications and consist of four components:

- **Backend service:** A local FastAPI backend wraps a pretrained CEFR classification model (`theluantran/cefr-bert-classifier`, a fine-tuned XLM-RoBERTa model), exposing a single `POST /classify` endpoint. The server is loaded only once at the server startup and returns probability scores per class across each of the five CEFR scores, (A1, A2, B1, B2, C1/C2).
- **Vanilla JavaScript implementation:** In order to establish a performance baseline against which framework-based implementations can be compared, a frontend implementation with no framework was created. Instead, this implementation uses manual DOM manipulation and the native `fetch` API, by doing so this implementation eliminates the abstraction and bundle-size overhead introduced by frontend frameworks, therefore isolating framework-specific overhead in results.
- **React implementation:** A second front-end implementation was developed with React (via Vite). Importantly, it contains a functionally identical UI to the other implementations. React uses `useState` hooks for state management for asynchronous request lifecycles, namely loading/success/error states.
- **Angular implementation:** A third front-end implementation was built with Angulars standalone components, simplifying dependency management by removing the traditional ngModule wrapper. This implementation uses Angular signals for reactive state management such as reactive forms.

All three frontend implementations share a single `styles.css` file, that is reused in order to preserve structure and visuals between the applications. This design choice ensures that all UI related variables are controlled and do not affect results. This allows framework specific overhead to be isolated in future performance analysis.

[1] https://github.com/kav51/Masters-Diss-App

Prior to conducting any performance tests, basic corrrectness tests were carried out to validate backend reliability. This step ensures that the classification model consistently produces correct outputs, avoiding mislabeling. This was done by submitting text samples at different CEFR levels. A short sentence containing little grammatical error was classified as A1 with a confidence level of 99.98%, whilst a longer entry was consistently classified as B2 across all frontend implementations. This confirms both that the model-serving pipeline functions correctly and that classification behaviour is consistent across all three frameworks when given identical input.

### B. Progress Against Timetable

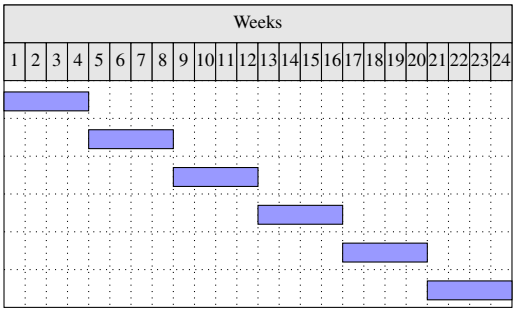


Fig. 1: Project Timeline

The above Fig.1 showcases the original 24 week project timeline, highlighting 6 stages. Currently at the time of writing this Interim report, we fall in between the boundaries of the third stage, weeks 9-12. Below describes how progress made to date maps closely to the first stages.

- **Weeks 1–4** (literature review, application specification, HuggingFace API validation, project scaffolding) have been completed.
- **Weeks 5–8** (Vanilla JS and React implementations) have been completed, with pixel-diff style visual parity maintained through the shared `styles.css` file described above.
- **Weeks 9–12** (Angular implementation, full validation across all three versions) have been completed, including the correctness testing described in Section III-A.

So far, the project is currently on schedule. There is only one deviation from the original plan, that being the failed integration method of the HuggingFace API, described above in section III. This was promptly resolved without deviating from the timeline, as the self-hosted backend was implemented within the same stage. The next stage which involves building and validating a Playwright-based benchmarking harness, capable of automating test exectution across all frontend implementations under simulated classrooms environments, before executing the full test criteria and collecting performance analytics. This stage will begin immediately after the submission of this Interim report.

### C. Unexpected Problems Encountered

During the course of development, there were two standout implementation issues that were encountered and resolved.

Initially, the original implmenetation plan assumed that the CEFR classifier would be accessible through HuggingFace's hosted API. However during implementation it was discovered that this specific model (`theluantran/cefr-bert-classifier`) is not deployed by any HuggingFace Inference Provider. In order to resolve this, the model was run locally through a self-hosted FastAPI backend, which worked in the favour of the development process as it removed an external network dependancy from the measured request path, improving experimenal control.

Secondly, after submitting the text for classification for the Angular implementation, it appeared to hang indefinitely despite the backend responding in under 300ms. This was confirmed in Browser Network Inspection in Developer tools, which revealed that the delay was in the user-interface failing to re-render. After tracing the issue back it was identified as Angulars zoneless change-detection model not detecting the state changes that were made inside asynchronous functions when using plain class properties. he issue was resolved by migrating component state to Angular Signals, which explicitly notify Angular's rendering engine of state changes.

## IV. What Are You Doing Next?

The remaining work is organised into three phases, mapped onto the final three quarters of the original 24-week timeline shown in Fig.1.

### A. Weeks 13-16: Benchmarking Harness

A playwright based test harness will be used in order to automate the benchmarking process across all three implementations. To simulate resource constrained classrom environments, the harness will interface with Chrome DevTools in order to apply CPU throttling at 4x and 6x, this is to represent the single core performance of entry-level Chromebook processors. Additionally, network conditions are capped at 5Mbs with 100ms RTT, this is to more accurately simulate congested school WIFI. Each test scenario is executed 30 times per frontend implementation and end environment combination. The median and interquartile range will also be reported so that results are robust to outliers.

Four metrics are captured for each run:

- **Bundle size:** total compressed JavaScript payload (gzip), analysed using `webpack-bundle-analyzer`, measuring network cost on constrained school bandwidth.
- **Time-to-Interactive (TTI):** Lighthouse-reported TTI, assessing initial application readiness under constrained hardware profiles.
- **Time-to-Result (TTR):** end-to-end latency from form submission to rendered classification result, measured via

the Performance API instrumentation already present in each implementation.
- **Peak heap memory:** maximum JavaScript heap usage during a session, captured via the Chrome DevTools Protocol, to assess memory pressure on low-RAM devices.

### *B. Weeks 17-20: Statistical Analysis*

During this stage, the raw data collected from previous stages will be analysed using various statstical methods in order to determine whether the observed differences between frameworks are significant or not. Kruskal-walis will be applied across the three frontend implementations for each metric and environment combination, with Bonferroni correction applied to control for multiple comparisons. Where significant effect is found, pairwise Mann-Whitney U tests are utilised in order to identify which framework pairs differ, and Cohens d will be calculated to assess the practical size of effects, not merely its statistical significance. In addition to this, results will also be examined for a degradation gradient, which is to check whether the performance gap between frameworks widens as hardware and network constraints intensify, linking back to the projects research question.

### *C. Weeks 21-24: Dissertation Write-Up*

The final phase will be focusing on the dissertation report, more specifically incorporating the completed results, statistical analysis, and including a critical discussion of findings, limitations, and their implications for framework selection in educational contexts. It is during this phase that I plan to communicate more thoroughly with my supervisor, planning regular review cycles to ensure that the final submission meets the requires standard ahead of the deadline.

## V. Appraisal and Reflection

Overall, progress has moved forwards as expected, with all three frontend implementations and backend behaving consistently. This gives me confidence that the benchmarking stage ahead will also be as consistent. Achieving this despite a lack of prior knowledge on certain tools such as FastAPI has required a steep yet rewarding learning curve, and the time invested in learning these technologies have been justified by the stability of each of the implementations.

In hindsight, the two implementation issues encountered that were discussed above were just as valuable as they were a thorn in the projects side. Both of these errors required methodically diagnosing the problem rather than guesswork. This is a lesson that I believe I can directly apply into the later stages, where careful debugging may be required when analysing and interpreting outliers.

Time management is another skill that I sharpened over the last couple of weeks, and it is also another skill I will be be utilising in future stages, as the benchmarking and statistical analysis stages are more time constrained and dependancy heavy than the stages completed so far.

## VI. Project Management

So far, project management has been self directed, structured entirely around the 24 week timeline shown in Fig.1. Work has progressed stage by stage through the timeline. All source code is version controlled using Git and hosted on GitHub[2], with the initial repository structure and subsequent implementation work committed and pushed once each component was complete and tested locally.

## VII. AI Usage

Throughout the writing of this interim report, I have used AI in a limited capacity, specifically for minor sentence level refinements. These tools were occassionally requested to rephrase or improve clarity, but all substational content such as technical decision making and analytics remain my own. AI generated suggestions were also adapted after being critically reviewed.

## References


[1] S. Krause, "js-framework-benchmark," https://github.com/krausest/js-framework-benchmark, 2025, accessed: July 14, 2026.

[2] M. Piastou, "Comprehensive performance and scalability assessment of front-end frameworks react angular and vuejs," ResearchGate, 2024, accessed: July 24, 2026. [Online]. Available: https://www.researchgate.net/publication/384307903_Comprehensive_Performance_and_Scalability_Assessment_of_Front-End_Frameworks_React_Angular_and_Vuejs

[3] A. Pandey, "Breaking down complexity: Maml's impact on web page optimization in developing regions," NYUAD Capstone Project 2 Reports, Spring 2024, Abu Dhabi, UAE, 2024, accessed: July 24, 2026. [Online]. Available: https://yasirzaki.net/capstones/Ayush_capstone.pdf

[4] N. S. Lagutina, K. V. Lagutina, A. M. Brederman, and N. N. Kasatkina, "Text classification by cefr levels using machine learning methods and the bert language model," *Automatic Control and Computer Sciences*, vol. 58, no. 7, pp. 869–878, 2024, accessed: July 26, 2026. [Online]. Available: https://link.springer.com/article/10.3103/S0146411624700329


[2]https://github.com/kav51/Masters-Diss-App